Git & GitHub
— Pathways into Quantitative Aging Research

Welcome! This notebook gets you set up to work on the **PQAR** *Pathway into Quantitative Aging* Projects. By the end you'll be able to:

- Use a Linux **terminal** for the basics
- Install **Git** and connect your computer to **GitHub**
- Download the project, save your work, and share it online
- Work on **branches** and fix a **merge conflict** without panicking

We'll practice on a real sample repository: `BoyuFan1/PQAR_sample` (https://github.com/BoyuFan1/PQAR_sample).

Two words people mix up:

- **Git** = a program that runs on *your computer* and tracks changes to your files.
- **GitHub** = a *website* that stores a copy of your project online so you (and teammates) can share it.

We'll install Git in Section 2, connect to the PQAR_sample project in Section 3, and start working in Section 4.

How to use this notebook

The blue **code cells** below show commands to **type (or copy) into your terminal**

After the important commands, you'll see a **"Check"** box telling you roughly what should appear on your screen, so you can confirm it worked before moving on.

---

1. Terminal Operations

The terminal is a way to talk to the computer by **typing commands** instead of clicking.

Open it: press `Ctrl` + `Alt` + `T`, or open the applications menu, search "Terminal", and click it.

You'll see a line ending in `$`. That's the **prompt** — the computer waiting for you. Type a command, press **Enter** to run it. (Don't type the `$` itself; it just marks the start of the line.)

Try these five commands one at a time. Type each, press Enter, see what happens.

TERMINAL
```
pwd
```

`pwd` = "print working directory" — tells you where you are right now.

Check — something like:

```
/home/yourname
```

TERMINAL
```
ls
```

`ls` lists the files and folders here. It might be empty for now — that's fine.

TERMINAL
```
mkdir projects
```

`mkdir` makes a new folder called `projects`. No output means it worked — run `ls` again and you'll see it.

TERMINAL
```
cd projects
```

`cd` = "change directory" — steps *into* the folder. Your prompt now shows `.../projects`.

- `cd ..` goes back up one level
- `cd` by itself jumps to your home folder

```
TERMINAL
clear
```

`clear` tidies the screen (or press `Ctrl` + `L`). It doesn't delete anything.

Four things that trip up everyone — read these now

- **Pasting is different here.** `Ctrl + V` does **not** paste in a terminal. Use `Ctrl + Shift + V`, or **right-click → Paste**.
- **Tab finishes your typing.** Start a file or folder name and press `Tab` — it auto-completes. Fewer typos.
- **Up-arrow repeats.** Press the ↑ key to bring back your last command instead of retyping.
- **Passwords are invisible.** When something asks for a password (like installing Git next), nothing shows as you type — no dots, no stars. That's normal. Type it and press Enter.

---

2. Install Git and connect to GitHub

This section has the most steps, but you only do it once per computer. After it's done, sharing your work is a one-line command forever after.

2.1 Check whether Git is already installed

```
TERMINAL
git --version
```

Check — if you see a version number, Git is already installed. **Skip to 2.2.**

```
git version 2.43.0
```

If instead you see `Command 'git' not found`, install it with the two commands below.

```
TERMINAL
sudo apt update
sudo apt install git
```

`sudo` means "do this as an administrator," so it asks for your computer's password — **remember, the password is invisible as you type it**. If it asks you to confirm, type `Y` then Enter.

(On Fedora/RHEL machines the command is `sudo dnf install git` instead.)

When it finishes, confirm:

```
TERMINAL
git --version
```

Check:

```
git version 2.43.0
```

2.2 Tell Git who you are

Every save you make (a "commit") is labeled with your name and email. Set them once. **These two commands print nothing — silence means success.** Use the email on your GitHub account.

TERMINAL

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Set two helpful defaults (also silent):

TERMINAL

```
git config --global init.defaultBranch main
git config --global core.editor nano
```

Check that it all saved:

TERMINAL

```
git config --list
```

Check — you should see your settings listed:

```
user.name=Your Name
user.email=you@example.com
init.defaultbranch=main
core.editor=nano
```

2.3 See if you already have an SSH key

An SSH key is how GitHub recognizes your computer. First check whether one exists (so you don't overwrite it):

TERMINAL

```
ls ~/.ssh
```

Check:

- If you see `id_ed25519` and `id_ed25519.pub` → you already have a key, **skip to 2.5**.
- If you see `No such file or directory` → no key yet, continue to 2.4.

2.4 Make an SSH key

Type the command below, then **press Enter three times** to accept the defaults (default location, and an empty passphrase twice).

TERMINAL

```
ssh-keygen -t ed25519 -C "you@example.com"
```

Check — you'll see a few prompts (press Enter at each), then a confirmation and a little ASCII-art box:

```
Enter file in which to save the key (...):      <- press Enter
Enter passphrase (empty for no passphrase):     <- press Enter
Enter same passphrase again:                    <- press Enter
Your identification has been saved in /home/you/.ssh/id_ed25519
Your public key has been saved in /home/you/.ssh/id_ed25519.pub
The key's randomart image is:
+--[ED25519 256]--+
|       ..oo.     |
|      . o..      |
|   ... .  ..     |
+----[SHA256]-----+
```

Heads up: the passphrase prompts show nothing as you type — you're just pressing Enter, so it's fine. The art box is normal, not an error.

2.5 Copy your public key

TERMINAL

```
cat ~/.ssh/id_ed25519.pub
```

Check — one long line starting with `ssh-ed25519`:

```
ssh-ed25519 AAXC...long...blob... you@example.com
```

Copy the entire line, from `ssh-ed25519` all the way through your email. (It's the file ending in `.pub` — the *public* one — that you share. Never share the other file.)

2.6 Add the key to GitHub (in your browser)

1. Click your **avatar** (top-right) → **Settings**
2. In the left menu: **SSH and GPG keys**
3. Click **New SSH key**
4. **Title:** anything, e.g. `my-laptop`
5. **Key:** paste the whole line you copied
6. Click **Add SSH key**

2.7 Test the connection

TERMINAL

```
ssh -T git@github.com
```

The very first time, it asks whether to trust GitHub's server — **type `yes` and press Enter.**

Check — success looks like this:

```
Hi yourname! You've successfully authenticated, but GitHub does not provide shell access.
```

Heads up: the ending "*but GitHub does not provide shell access*" sounds like an error but it isn't — that whole line means it **worked**. You're connected. From now on, sharing your work needs no passwords.

---

3. The project on GitHub: `PQAR_sample`

For this tour we use a shared project called **PQAR_sample** — a sample for the *Pathway of Quantitative Aging Research* work. It already exists on GitHub:

`https://github.com/BoyuFan1/PQAR_sample`

When you create your *own* project later, you'd do it in the browser:

1. Click the **+** in the top-right → **New repository**
2. **Repository name:** e.g. `PQAR_sample` (no spaces; hyphens and underscores are okay)
3. **Description:** one line, optional — e.g. "Pathway of Quantitative Aging Research — sample"
4. **Visibility:** Public or Private
5. Check **Add a README file** (this gives your project a first file so you can download it right away)
6. **Add .gitignore:** pick Python so junk files aren't tracked
7. Click **Create repository**

To get the address for the next step: on the project page, click the green **Code** button, choose the **SSH** tab, and copy the line that starts with `git@github.com:`. For PQAR_sample that address is:

`git@github.com:BoyuFan1/PQAR_sample.git`

> A "repository" (or "repo") is just the name for a project folder that Git tracks.

4. Download the project to your computer

"Cloning" means downloading a copy. Go into the `projects` folder you made earlier, then clone PQAR_sample using its SSH address:

TERMINAL

```
cd ~/projects
git clone git@github.com:BoyuFan1/PQAR_sample.git
```

Check:

```
Cloning into 'PQAR_sample'...
remote: Enumerating objects: 3, done.
Receiving objects: 100% (3/3), done.
```

Now step into the project and look around:

TERMINAL

```
cd PQAR_sample
ls -la
```

You'll notice a hidden `.git` folder. That folder *is* what makes this a Git project — it quietly stores your whole history. Leave it alone; never edit it by hand.

(When you work on your own project, swap in your repo's SSH address instead of `BoyuFan1/PQAR_sample`.)

5. The everyday loop: edit → stage → commit → push

This four-step cycle is 90% of what you'll do all summer.

Step	Command	Plain English
1. Stage	`git add`	"I want to save these files"
2. Commit	`git commit`	"Save a snapshot, with a note"
3. Push	`git push`	"Upload it to GitHub"
(Start of day)	`git pull`	"Download any new changes first"

5.1 Make a change

TERMINAL

```
echo "# PQAR_sample" > README.md
echo "Notes for the quantitative aging analysis." >> notes.md
```

Ask Git what changed:

TERMINAL

```
git status
```

Check — it reports a modified file and a new ("untracked") one, shown in red:

```
Changes not staged for commit:
        modified:   README.md
Untracked files:
        notes.md
```

5.2 Stage the changes

TERMINAL

```
git add notes.md README.md
```

Run `git status` again — the files are now green under "Changes to be committed." (Tip: `git add .` stages *everything* that changed.)

5.3 Commit (save a snapshot)

Write a short message saying what you did, in the imperative: "Add...", "Fix...".

TERMINAL

```
git commit -m "Add notes and update README"
```

Check:

```
[main 1a2b3c4] Add notes and update README
 2 files changed, 2 insertions(+)
```

5.4 Push (upload to GitHub)

TERMINAL

```
git push
```

Check — ends with a line like `main -> main`. Now refresh your project page on GitHub in the browser: your files are there!

5.5 See your history, and pull updates

TERMINAL

```
git log --oneline
```

Check — one line per snapshot (press `q` to exit this view):

```
1a2b3c4 (HEAD -> main, origin/main) Add notes and update README
abc1234 Initial commit
```

When a teammate has uploaded changes, download them **before** you start working:

TERMINAL

```
git pull
```

Golden habit: `git pull` when you sit down to work, `git push` when you finish. This alone prevents most problems.

---

6. Branches

A **branch** is a safe side-copy where you can experiment without touching the main version. `main` stays the trustworthy copy; you try things on a branch, then merge them back when ready.

Create a branch and switch onto it:

TERMINAL

```
git switch -c add-aging-clock
```

Check: `Switched to a new branch 'add-aging-clock'`. See where you are (the `*` marks your branch):

TERMINAL

```
git branch
```

Do some work and commit it on the branch:

TERMINAL

```
echo "import pandas as pd  # load aging biomarker data" > aging_clock.py
git add aging_clock.py
git commit -m "Start aging-clock script"
```

Bring your work back into `main`:

TERMINAL

```
git switch main
git merge add-aging-clock
```

Check — a clean merge looks like:

```
Fast-forward
 aging_clock.py | 1 +
```

That's the smooth case. Next, we'll create a *conflict* on purpose so it doesn't scare you when it happens for real.

---

7. Fixing a merge conflict

A **conflict** happens when two branches change the **same line of the same file** in different ways, and Git can't decide which one wins. It's not a crash — Git is just asking *you* to choose. Let's make one happen with a realistic PQAR example: two collaborators set the **same parameter** of the aging-clock model to different values in `config.txt`.

7.1 Set up a config file on `main`

TERMINAL

```
git switch main
echo "learning_rate = 0.01" > config.txt
git add config.txt
git commit -m "Add aging-clock config"
```

7.2 On a new branch, change the line one way

Since `config.txt` is a single line, the simplest way to "edit" it is to overwrite it with `echo` (the `>` replaces the file's contents). Create a branch and set a faster rate:

TERMINAL

```
git switch -c faster-lr
echo "learning_rate = 0.1" > config.txt
git commit -am "Use a faster learning rate"
```

7.3 Back on `main`, change the same line a different way

TERMINAL

```
git switch main
echo "learning_rate = 0.001" > config.txt
git commit -am "Use a slower learning rate"
```

7.4 Merge — and watch the conflict appear

TERMINAL

```
git merge faster-lr
```

Check — this is expected:

```
CONFLICT (content): Merge conflict in config.txt
Automatic merge failed; fix conflicts and then commit the result.
```

7.5 Look at the conflict

TERMINAL

```
cat config.txt
```

Check — Git inserted markers showing both versions:

```
<<<<<<< HEAD
learning_rate = 0.001
=======
learning_rate = 0.1
>>>>>>> faster-lr
```

Reading it top to bottom:

- `<<<<<<< HEAD` → start of **your current branch's** version (here: `0.001`)
- `=======` → the divider between the two versions
- below the divider → the **incoming** version from the branch you're merging (here: `0.1`)
- `>>>>>>> faster-lr` → end of the incoming version

7.6 Resolve it

Open the file and **delete the markers, keeping only the content you want** — one side, the other, or something new. Here we'll compromise on a middle value.

TERMINAL

```
nano config.txt
```

Edit the file so it contains **exactly** this and nothing else:

```
learning_rate = 0.05
```

Make sure **none** of the marker lines (`<<<<<<<`, `=======`, `>>>>>>>`) are left — leftover markers are the most common mistake.

In nano: edit the text, then `Ctrl + O` and Enter to save, `Ctrl + X` to exit.

7.7 Finish the merge

Staging the file tells Git "I've handled it," then commit to complete the merge.

TERMINAL

```
git add config.txt
git commit -m "Resolve conflict: settle on learning_rate = 0.05"
git push
```

Done — you've resolved a merge conflict.

Escape hatch

If a merge ever feels like a mess and you want to undo it and start over:

TERMINAL

```
git merge --abort
```

This rewinds to right before the merge, as if it never happened. Good to know so you never feel stuck.

Avoiding conflicts: `git pull` before you start and before you push, commit small and often, and try not to edit the same file as a teammate at the same time.

---

8. When something goes wrong

Message you see	What it means	What to do
`bash: git: command not found`	Git isn't installed	Do step 2.1
`fatal: not a git repository`	You're not inside your project folder	`cd` into the folder (the one with `.git`)
`Author identity unknown`	Name/email not set	Re-do step 2.2
`Permission denied (publickey)`	SSH key not connected	Re-do steps 2.4–2.7
`Updates were rejected ... fetch first`	GitHub has changes you don't have yet	Run `git pull`, fix anything, then `git push`
`nothing to commit, working tree clean`	Nothing changed, or you forgot `git add`	Edit a file, then `git add`, then commit
An editor opens after `git commit`	You forgot the `-m "message"` part	Type a message, save & exit (`Ctrl+O`, Enter, `Ctrl+X`)
Conflict markers still in your file	You missed a `<<<<<<<` line	Edit the file, remove all markers, `git add`, commit

---

Cheat sheet

```
# terminal basics
pwd                      # where am I
ls                       # what's here
cd <folder>              # go in   (cd .. = up,  cd = home)
mkdir <folder>           # make a folder
# paste = Ctrl+Shift+V  |  Tab = autocomplete  |  Up-arrow = last command

# one-time setup
sudo apt install git
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
ssh-keygen -t ed25519 -C "you@example.com"     # press Enter x3
cat ~/.ssh/id_ed25519.pub                       # copy this to GitHub
ssh -T git@github.com                           # test it

# start a project (clone from github.com)
git clone git@github.com:BoyuFan1/PQAR_sample.git
cd PQAR_sample

# the everyday loop
git pull                 # before you start
git status               # what changed?
git add <file>           # stage  (git add . = all)
git commit -m "message"  # save a snapshot
git push                 # upload to GitHub

# branches
git switch -c <name>     # create + switch
git switch <name>        # switch to existing
git merge <name>         # merge a branch into this one

# conflicts
# 1) git merge <branch>  ->  CONFLICT
# 2) edit the file, delete the <<<<<<< ======= >>>>>>> markers
# 3) git add <file>
# 4) git commit
git merge --abort        # undo a messy merge
```